

Name : Sakshi Dattu Bodkhe

Class: AIDS

Roll No : SYA-14

Sub : Python For Data Science

Aim: 8. Implement and analyze various methods of studying correlation and regression including Karl Pearson's coefficient and simple linear regression.

Theory:

1. What is correlation? How is it different from regression?

- **Correlation** measures the strength and direction of the linear relationship between two variables. It is quantified by a correlation coefficient (like Karl Pearson's r), which ranges from (-1) to $(+1)$.
- **Regression** goes further: it explains how one variable (dependent) changes when another variable (independent) changes. It provides an equation to predict values.

Key difference:

- Correlation is **symmetric** (relationship between X and Y is the same as between Y and X).
- Regression is **asymmetric** (predicts Y from X, not the other way around).

✓ 2. What is simple linear regression? Explain with example.

- **Definition:** Simple linear regression models the relationship between two variables using a straight line: $[Y = \beta_0 + \beta_1 X + \epsilon]$ where (Y) is the dependent variable, (X) is the independent variable, (β_0) is the intercept, (β_1) is the slope, and (ϵ) is the error term.
- **Example:** Suppose we want to predict a student's exam score ((Y)) based on hours studied ((X)). If regression gives: $[\text{Score} = 40 + 5 \cdot (\text{Hours Studied})]$ then studying 6 hours predicts a score of $(40 + 5 \cdot 6 = 70)$.

3. How do you calculate Pearson correlation in Python?

```
import numpy as np
import pandas as pd
```

```
# Example data
x = [10, 20, 30, 40, 50]
y = [15, 25, 35, 45, 55]

# Using numpy
corr_np = np.corrcoef(x, y)[0, 1]

# Using pandas
df = pd.DataFrame({'x': x, 'y': y})
corr_pd = df['x'].corr(df['y'])

print("Pearson correlation (numpy):", corr_np)
print("Pearson correlation (pandas):", corr_pd)
```

Both methods will give a correlation coefficient close to **1.0**, showing a strong positive linear relationship.

4. What is multicollinearity? Does it affect simple regression?

- **Multicollinearity** occurs when independent variables in a regression model are highly correlated with each other.
- It inflates variance of coefficients, making them unstable and hard to interpret.
- In **simple regression** (only one independent variable), multicollinearity does **not** occur because there's only one predictor. It becomes an issue in **multiple regression**.

5. A dataset shows strong correlation but poor prediction accuracy. Why?

Possible reasons:

- **Overfitting or underfitting:** The model may not generalize well beyond the training data.
- **Non-linearity:** Correlation measures linear relationships. If the true relationship is non-linear, correlation can be high but regression predictions poor.
- **Outliers:** Strong correlation may be driven by a few extreme values, reducing predictive accuracy.
- **Data leakage or irrelevant variables:** Correlation doesn't guarantee causation. The predictor may not truly explain the dependent variable.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Dataset
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1) # Hours studied
Y = np.array([40, 50, 60, 70, 80])         # Exam scores

# 1. Pearson correlation
corr = np.corrcoef(X.flatten(), Y)[0, 1]
print("Pearson correlation coefficient:", corr)

# 2. Fit simple linear regression
model = LinearRegression()
model.fit(X, Y)

# Regression parameters
intercept = model.intercept_
slope = model.coef_[0]
print(f"Regression equation: Score = {intercept:.2f} + {slope:.2f} * Hours")

# 3. Predictions
Y_pred = model.predict(X)

# 4. Plot scatter plot with regression line
plt.scatter(X, Y, color='blue', label='Actual data')
plt.plot(X, Y_pred, color='red', label='Regression line')
plt.xlabel("Hours Studied")
plt.ylabel("Exam Score")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()
```

Pearson correlation coefficient: 1.0

Regression equation: $\text{Score} = 30.00 + 10.00 * \text{Hours}$

Simple Linear Regression

